“Linux-Based Alarm Clock System”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

PARAM MANASVI	**2520030072**
BURLA VIDA SWETHA	**2520030100**

Under the Esteemed Guidance of

Dr. K. Hema
Assistant Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOL



Declaration

The Project Report entitled **"Linux-Based Alarm Clock System"** is a record of Bonafide work of **BURLA VIDA SWETHA - 2520030100 , PARAM MANASVI – 2520030072** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

PARAM MANASVI	**2520030072**
BURLA VIDA SWETHA	**2520030100**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled **"Linux-Based Alarm Clock System"** is a bonafide work done and submitted by of **BURLA VIDA SWETHA - 2520030100 , PARAM MANASVI – 2520030072,** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2024-2025.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator and my guide Dr. K. Hema **,** Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)**, Head of the ***Department for Computer Science Engineering*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

PARAM MANASVI **2520030072**
BURLA VIDA SWETHA **2520030100**

Abstract

Setting an alarm looks simple, but it depends on several core services of an operating system: process creation, timers, signals, scheduling and inter-process communication. Many simple implementations wait by repeatedly checking the clock, which wastes CPU time and leaves the user with no clean way to cancel. Our project, the Design and Development of a Linux-Based Alarm Clock System Using Processes, Signals and Timers, addresses this properly. It lets a user schedule an alarm for a chosen delay or clock time, continue other work while waiting, and then be notified or cancel the alarm, all through a web interface.

The system is written in C and runs entirely in Linux user space. A small C HTTP server accepts requests from the browser and creates a separate child process with fork() for every alarm. Each worker arms a kernel timer with alarm(), sleeps in sigsuspend() so that it uses no CPU, and wakes on SIGALRM when the time arrives. It then notifies the user through the terminal, a Linux desktop notification (notify-send) and a banner with a repeating sound in the web page. An active alarm can be cancelled with SIGUSR1 and a ringing alarm stopped with SIGUSR2, and finished workers are reaped through SIGCHLD so that no zombie processes remain. A shared SQLite database, used in WAL mode, acts as the inter-process communication and persistence layer between the server and the workers.

The most distinctive feature is the live CPU scheduler view. Every worker logs its real state changes with the CPU core and a microsecond timestamp, and the server reads the Linux /proc filesystem, so the browser shows each alarm moving through the New, Wait queue, Ready queue, CPU and Exited states together with the run-queue delay, timer latency and context switches. In testing on Ubuntu, the project built without compiler warnings, a 6-second alarm fired with a timer latency of about 153 microseconds while the waiting process used no CPU time, and the cancel, stop and invalid-input cases behaved correctly. In summary, the project delivers a complete, efficient and observable alarm clock that demonstrates the main concepts of operating systems and systems programming.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		8–11
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	OS Concepts and Background Study		12–15
		2.1 Operating System Concepts Related to the Project	
		2.2 Existing Methods / Algorithms	
		2.3 Proposed Approach	
		2.4 Advantages of the Proposed Approach	
3	System Requirements		16–17
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment – Ubuntu / WSL	
		3.4 Development Tools and Compilers	
4	Technology and Tools Used		18–20
		4.1 C Programming	
		4.2 GCC Compiler	
		4.3 Linux System Calls	
		4.4 POSIX Threads / Synchronization	
		4.5 Linux Commands and Utilities	
		4.6 Other Tools / Libraries Used	
5	System Architecture and Design		21–27
		5.1 Overall System Architecture	
		5.2 System Workflow	
		5.3 Flowchart / Data Flow	

		5.4 Module Description	
		5.5 Input and Output Design	
6	**Implementation**		**28–34**
		6.1 Module-wise Implementation	
		6.2 Algorithm / Pseudocode	
		6.3 Core C Implementation	
		6.4 Linux System Calls / APIs Used	
		6.5 Process / Thread Implementation	
		6.6 IPC / Synchronization Implementation	
		6.7 Memory Management Implementation	
		6.8 Optional User Interface Implementation	
7	**Execution and Demonstration**		**35–43**
		7.1 Compilation in Ubuntu	
		7.2 Execution Commands	
		7.3 Sample Inputs and Outputs	
		7.4 Screenshots of Execution	
		7.5 Demonstration of OS Concepts	
8	**Conclusion**		**44–45**
		8.1 Summary of the Project	
		8.2 Objectives Achieved	
		8.3 Results Achieved	
		8.4 Skills and Learning Outcomes	
9	**References**		**46–47**
		9.1 Books	
		9.2 Research Papers	
		9.3 Linux / GNU Documentation	
		9.4 Online Resources	

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

An alarm clock looks like one of the simplest programs a computer can run: wait until a chosen moment, then tell the user. Behind that simple behaviour, however, sits a good part of what an operating system is responsible for. The program must be created as a process, it must be able to sleep without wasting processor time, it must be woken at the right moment by a timer, it must react to asynchronous events such as "cancel" or "stop", and it must share its state with other programs. Each of these needs is served by a core operating-system service: process management, timers, signals, scheduling and inter-process communication.

This project, the **Alarm Clock System**, was built for the Operating Systems and Systems Programming scenario: *schedule an alarm for a specific time, keep doing other work while waiting, and get notified (or cancel) when the time comes.* Instead of a purely command-line demonstration, the system is delivered as a small web application so that the behaviour of the operating system can be seen as well as read about. The whole back end, including the web server, is written in C and runs in Linux user space. No kernel modification is required, and the only external library is SQLite.

Every alarm that a user sets is handled by its **own Linux process**. The process is created with `fork()`, sleeps with `sigsuspend()` so that it consumes no CPU while it waits, is woken by the `SIGALRM` signal when the timer set with `alarm()` expires, and can be cancelled or stopped by the parent using `SIGUSR1` and `SIGUSR2`. A live "CPU scheduler" panel in the browser then shows each alarm moving through the classic process states (New, Waiting, Ready, Running, Exited), driven by what the worker process really did and by data read from the Linux `/proc` filesystem.

1.2 Problem Statement

Users frequently need to be reminded of something at a particular time while they continue with other work. A naive solution is a program that sits in a loop and keeps checking the clock. Such a loop wastes CPU time, delays other programs, and gives the user no clean way of cancelling the wait. A better solution must satisfy the following requirements, all of which are operating-system problems:

- accept an alarm time (as a delay or as a clock time) from the user;
- create a separate process to manage each alarm so that several alarms can exist independently;
- use timers and signals to generate the alarm event instead of polling the clock;

- suspend the waiting process so that it uses no CPU time until the event happens;
- handle the alarm signal and notify the user in an unmistakable way;
- allow an active alarm to be cancelled, and a ringing alarm to be stopped;
- share alarm information between the server and the alarm processes safely; and
- make the internal behaviour (process states, CPU core used, scheduling delay) visible for learning and demonstration.

The problem addressed by this project is therefore: **to design and implement, using only standard Linux system calls, an alarm clock that is efficient (no busy-waiting), concurrent (many alarms at once), controllable (cancel / stop), persistent (stored in a database) and observable (scheduler view) through a web interface.**

1.3 Scope of the Project

The scope of the project covers the following functionality.

- **Alarm scheduling:** the user enters a label and either a delay (hours, minutes, seconds) or a clock time. Clock times that have already passed today are scheduled for the next day.
- **Process-per-alarm model:** each alarm is owned by a dedicated child process created by `fork()`.
- **Timer and signal handling:** `alarm()`, `SIGALRM`, `SIGUSR1`, `SIGUSR2` and `SIGCHLD` are used with `sigaction()`, `sigprocmask()` and `sigsuspend()`.
- **Notification:** a message on the server console, a Linux desktop notification through `notify-send`, and a red banner, toast message and repeating alarm sound in the web page.
- **Control:** cancel an active alarm, stop a ringing alarm, and clear finished alarms.
- **Persistence and IPC:** alarms and scheduler events are stored in an SQLite database that is shared by the server and all worker processes.
- **Observation:** a scheduler panel, a live process table and an event log, based on real timestamps, CPU core numbers and `/proc` data.

The following are outside the scope of the project: modifying the Linux kernel, recurring (daily / weekly) alarms, user accounts and authentication, encrypted (HTTPS) communication, and re-attaching to alarm processes after the server itself has been stopped. If the server is restarted, alarms whose worker processes no longer exist are marked as cancelled at start-up.

1.4 Objectives of the Project

1. To study and apply the operating-system concepts of process creation, process states, signals, timers, scheduling and inter-process communication in a working application.
2. To implement one process per alarm using `fork()` and to manage the lifetime of these processes without leaving zombie processes.
3. To generate the alarm event with the kernel timer (`alarm()` and `SIGALRM`) and to wait without busy-waiting using `sigsuspend()`.
4. To allow cancellation and dismissal of alarms through signals (`SIGUSR1`, `SIGUSR2`) sent with `kill()`.
5. To use a shared SQLite database as the inter-process communication and persistence layer between the server and the worker processes.
6. To build a lightweight HTTP server in C, with a REST-style JSON interface, and a browser front end using HTML, CSS and JavaScript.
7. To expose scheduler activity (core used, run-queue delay, timer latency, context switches) so that the operating-system behaviour can be demonstrated.

1.5 Importance of the Application

The application is important from two points of view. For a **user**, it is a dependable, low-overhead reminder tool: alarms cost almost nothing while waiting because the processes are blocked inside the kernel, they fire with a measured latency of the order of microseconds to a few hundred microseconds, and they can be controlled from any browser.

For a **student of operating systems**, it is a compact, complete and runnable example that connects theory with practice. Concepts that are usually met only in textbook diagrams, such as the five-state process model, the wait queue and the ready queue, signal delivery, context switches and zombie processes, can be observed on a real system using commands like `ps --forest` and through the live panels of the web page. The code base is small (about 1,400 lines of source) and heavily commented, which makes it practical to read in full.

1.6 Target Users / Audience

Audience	How they use the system
Students of Operating Systems	Study and demonstrate process creation, signals, timers and scheduling on a real Linux machine.
Faculty and evaluators	Verify the use of system calls and the correctness of process life-cycle handling during project review.

Audience	How they use the system
General Linux users	Set quick timers and reminders from the browser, with a desktop notification when the alarm rings.
Systems programmers	Use the code as a reference for a signal-driven, process-per-task design with SQLite-based IPC.

CHAPTER -2 OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

The project is built on a small set of operating-system concepts. Each of them is described below together with the place where it is used in the system.

Process and process creation

A process is a program in execution with its own address space, process identifier (PID) and resources. On Linux a new process is created with the `fork()` system call, which duplicates the calling process. The parent receives the child's PID and the child receives 0. In this project the server forks once for every alarm; the child runs `run_alarm_worker()` and never returns to the server loop.

Process states and the scheduler

During its life a process moves between states: **New** (just created), **Ready** (runnable, waiting in the run queue for a CPU), **Running** (executing on a CPU), **Waiting / Blocked** (sleeping until an event) and **Terminated**. The scheduler decides which ready process runs next and on which core. The time a process spends ready but not yet running is the run-queue delay. The alarm worker deliberately spends almost its whole life in the Waiting state, and the system records every transition, so the web page can replay it.

Signals

A signal is a software notification delivered asynchronously to a process. A process may install a handler for a signal, block it, or leave the default action. The following signals are central to the project.

Table 2.1: Signals used in the project

Signal	Sent by	Meaning in this project
`SIGALRM`	Kernel timer, after `alarm(n)` expires	The alarm time has been reached; wakes the worker from `sigsuspend()`.
`SIGUSR1`	Server, using `kill(pid, SIGUSR1)`	Cancel an alarm that has not fired yet.
`SIGUSR2`	Server, using `kill(pid, SIGUSR2)`	Stop (dismiss) an alarm that is ringing.
`SIGCHLD`	Kernel, when a child exits	The server reaps finished workers with `waitpid()` so no zombies remain.
`SIGPIPE`	Kernel, writing to a closed socket	Ignored so that the server does not die when a browser disconnects.

Timers and suspension without busy-waiting

The `alarm(seconds)` call asks the kernel to deliver `SIGALRM` to the calling process after the given number of seconds. The process then calls `sigsuspend()`, which atomically replaces the signal mask and puts the process to sleep until a signal arrives. While asleep the process is removed from the run queue and consumes no CPU. Using `sigprocmask()` to block the signals first and `sigsuspend()` to wait avoids the classic lost-wake-up race in which a signal arrives between checking a flag and going to sleep.

Zombie processes and reaping

When a child terminates it remains in the process table as a zombie until the parent collects its exit status with `wait()` or `waitpid()`. The server installs a `SIGCHLD` handler that calls `waitpid(-1, NULL, WNOHANG)` in a loop, so every finished alarm process is cleaned up immediately.

Inter-process communication

The server and its workers need to share information: the server must know which PID owns which alarm, and the workers must publish status changes and scheduler events. Signals carry commands from parent to child, while a shared **SQLite database file** carries data in both directions. Because several processes write to it, the database runs in WAL (write-ahead logging) mode with a busy time-out.

The /proc filesystem and sockets

`/proc` is a virtual filesystem through which the kernel exposes process and CPU information. The project reads `/proc/stat` (per-core usage), `/proc/loadavg`, `/proc/<pid>/stat` (state, last CPU, CPU time), `/proc/<pid>/status` (context switches), `/proc/<pid>/wchan` (the kernel function a sleeping process is blocked in) and `/proc/self/schedstat` (run-queue wait time). The server itself communicates with browsers through TCP sockets created with `socket()`, `bind()`, `listen()` and `accept()`.

Table 2.2: Mapping of OS concepts to implementation

OS concept	System call / interface	Where used
Process creation	`fork()`	`handle_post_alarms()` in `server.c`
Timer	`alarm()`	`run_alarm_worker()` in `alarm_worker.c`
Signal handling	`sigaction()`	Worker handlers; server `SIGCHLD` handler
Blocking / waiting	`sigprocmask()`, `sigsuspend()`	Worker waits twice: before and after ringing
Process control	`kill()`	Cancel (`SIGUSR1`) and dismiss (`SIGUSR2`)

OS concept	System call / interface	Where used
Process cleanup	`waitpid()`, `exit()`	`SIGCHLD` handler and worker exit
Scheduling information	`sched_getcpu()`, `/proc`	Worker events and `sysinfo.c`
IPC / persistence	SQLite file in WAL mode	`db.c`, shared by all processes
Networking	Sockets	Server accept loop and `http.c`

2.2 Existing Methods / Algorithms

Several approaches exist for running an action at a later time. They differ in how they wait, how much they cost while waiting, and how well they can be controlled.

Table 2.3: Comparison of existing methods

Method	How it works	Limitations for this problem
Busy-wait / polling loop	Loop repeatedly reading the clock until the target time.	Burns CPU at 100% of a core; slows other work; hard to cancel cleanly.
`sleep()` in the foreground	Block the program for n seconds.	The program cannot do anything else; cancelling kills the whole program.
`cron` / `at` jobs	A system daemon runs a command at a scheduled time.	Minute granularity for cron; no live status, no cancel-from-browser, no per-alarm process visibility.
Threads with timers (`pthread`, `timerfd`)	One process, several threads or one event loop.	Signals and `alarm()` are process-wide; a fault in one thread can affect all alarms; harder to show per-alarm process states.
Desktop clock applications	Graphical programs using toolkit timers.	Hide the underlying OS mechanisms; not usable as a teaching tool.

2.3 Proposed Approach

The proposed approach is a **process-per-alarm, signal-driven design** supervised by a small C server:

1. The browser sends the alarm label and the delay to the server as JSON (`POST /api/alarms`).
2. The server stores the alarm in SQLite and calls `fork()`. The child becomes the alarm worker; the parent records the child's PID.
3. The worker installs signal handlers, blocks the signals, arms `alarm(delay)` and calls `sigsuspend()`. It now sleeps with zero CPU use.
4. When the timer expires the kernel sends `SIGALRM`; the worker wakes, the handler records the exact time, and the worker marks the alarm as ringing and notifies the user.

5. The worker then blocks again in `sigsuspend()` until the user presses **Stop**, which makes the server send `SIGUSR2`. A **Cancel** before the alarm fires makes the server send `SIGUSR1` instead.
6. The worker updates the database, logs its final events and calls `exit(0)`. The parent reaps it through the `SIGCHLD` handler.

At every step the worker writes an event (type, state, PID, CPU core, microsecond timestamp) to the `events` table. The browser polls these events and replays them as chips moving between the New, Wait queue, Ready queue, CPU cores and Exited panels.

2.4 Advantages of the Proposed Approach

- **No busy-waiting:** a waiting alarm is blocked in `sigsuspend()`; the live process table shows it in the sleeping state with 0 ms CPU time.
- **Isolation:** every alarm has its own address space and PID. A fault in one worker cannot corrupt another alarm or the server.
- **Precise timing:** the kernel timer delivers `SIGALRM` directly; the measured timer latency in testing was about 150 microseconds.
- **Simple control:** cancelling or stopping an alarm is one `kill()` call on a known PID.
- **Clean life cycle:** the `SIGCHLD` reaper prevents zombie processes; start-up cleanup marks orphaned alarms as cancelled.
- **Educational transparency:** real timestamps, core numbers and `/proc` data make scheduling visible.
- **Small dependency footprint:** only the C standard library, POSIX / Linux system calls and SQLite are needed.

CHAPTER -3 SYSTEM REQUIREMENTS

3.1 Hardware Requirements

The system is very light. The compiled server is about 94 KB and each alarm worker is a forked copy that sleeps almost all the time. The figures below are minimum and recommended values; the project was built and tested successfully on a single-core virtual machine, and a multi-core machine is only needed to see more than one CPU core in the scheduler panel.

Table 3.1: Hardware requirements

Component	Minimum	Recommended
Processor	1 core, x86-64	Multi-core (2 or more cores) to observe per-core scheduling
Memory (RAM)	512 MB	2 GB or more
Disk space	50 MB free (source, build output and database)	500 MB free
Display	Any screen able to run a web browser	1280 x 800 or larger for the full scheduler view
Audio output	Optional	Speakers or headphones for the alarm sound
Network	Loopback interface only (localhost)	LAN access if the page is opened from another device

3.2 Software Requirements

Table 3.2: Software requirements

Software	Version / package	Purpose
Operating system	Ubuntu 22.04 or later, Debian, or Ubuntu on WSL2 (tested on Ubuntu 24.04)	Provides the Linux system calls and `/proc`
C compiler	`gcc` (package `build-essential`)	Compiles the C source files with `-std=gnu11`
Build tool	`make`	Builds the project using the supplied Makefile
SQLite library	`libsqlite3-dev` (SQLite 3)	Database engine and C header `sqlite3.h`
Notification utility	`libnotify-bin` (provides `notify-send`)	Linux desktop notifications (optional)
Web browser	Any current browser (Chrome, Firefox, Edge)	Runs the user interface (HTML5, CSS3, JavaScript, Web Audio)
Utilities	`curl`, `ps`, `pstree`	Testing the REST API and observing the process tree

3.3 Operating System Environment – Ubuntu / WSL

The system uses Linux-specific facilities (`/proc`, `sched_getcpu()`, `notify-send`), so it must run on Linux. It runs natively on Ubuntu or Debian and also inside **Windows Subsystem for Linux (WSL2)** with an Ubuntu distribution. The environment is prepared with the following commands.

```
sudo apt update
sudo apt install -y build-essential libsqlite3-dev libnotify-bin
```

Listing 3.1: Installing the required packages

Two environment notes apply. First, `notify-send` needs a desktop session with a notification daemon; WSL does not have one by default, and a headless server will not show desktop pop-ups. In that case the alarm still works: the worker prints to the terminal and the web page displays the banner, toast and sound. Second, browsers only allow audio after the user interacts with the page, so pressing **Set alarm** or **Test sound** once enables the alarm sound.

3.4 Development Tools and Compilers

Table 3.3: Development tools

Tool	Use in the project
GNU C Compiler (`gcc`)	Compiles each `.c` file to an object file and links the final `alarm_server` executable.
GNU Make	Automates the build; targets `all`, `run` and `clean` are defined in the Makefile.
`gdb`	Source-level debugging; the build uses `-g` to include debug symbols.
`curl`	Calls the REST API from the terminal to test each endpoint without the browser.
`ps --forest`, `pstree`	Shows the parent server and the child alarm processes as a tree.
Text editor / IDE	Visual Studio Code or any editor for the C, HTML, CSS and JavaScript files.
Browser developer tools	Inspecting network calls and the JavaScript front end.

The compiler options used by the Makefile are `-Wall -Wextra -std=gnu11 -Iinclude -g`. The project compiles without warnings. The link step adds `-lsqlite3`.

CHAPTER -4 TECHNOLOGY AND TOOLS USED

4.1 C Programming

The whole back end is written in the **C programming language** (GNU C11 dialect). C was chosen because it gives direct access to the POSIX and Linux system calls that this project is about, with no runtime or framework hiding them. The source is organised as five implementation files and four headers.

Table 4.1: Source files and size

File	Lines	Responsibility
`src/server.c`	292	`main()`, TCP listener, request routing, `fork()`, `SIGCHLD` reaper
`src/alarm_worker.c`	153	Per-alarm child process: timer, signals, notification
`src/db.c`	271	SQLite persistence layer: schema, queries, JSON output
`src/http.c`	149	HTTP request reader, response writer, static file server, minimal JSON field extraction
`src/sysinfo.c`	145	Reads `/proc` to build the live CPU and process snapshot
`include/*.h`	4 files	Public interfaces: `db.h`, `http.h`, `alarm_worker.h`, `sysinfo.h`

Language features used include structures (`http_request_t`), function prototypes in headers, static module-level state, bounded string handling with `snprintf()`, `volatile sig_atomic_t` flags for signal handlers, and manual dynamic memory with `malloc()` and `free()`.

4.2 GCC Compiler

The GNU Compiler Collection (`gcc`) translates the C source into machine code. The Makefile defines `CC := gcc` and compiles every file in `src/` into `build/*.o`, then links them.

Table 4.2: GCC options used

Option	Meaning
`-std=gnu11`	C11 standard with GNU extensions (needed for `sched_getcpu()` and POSIX features)
`-Wall -Wextra`	Enable a broad set of compiler warnings; the build is warning-free
`-Iinclude`	Search the `include/` directory for project headers
`-g`	Include debugging information for `gdb`
`-lsqlite3`	Link against the SQLite library
`#define _GNU_SOURCE`	Source-level macro in `alarm_worker.c` and `sysinfo.c` that exposes `sched_getcpu()`

4.3 Linux System Calls

System calls are the entry points into the kernel. The project relies on the following ones; Section 6.4 explains where each is used.

Table 4.3: Linux system calls and library functions

Category	Calls
Process management	`fork()`, `exit()`, `waitpid()`, `getpid()`, `getppid()`, `execlp()`
Signals	`sigaction()`, `signal()`, `sigprocmask()`, `sigsuspend()`, `kill()`, `alarm()`
Time	`clock_gettime()`, `time()`
Scheduling information	`sched_getcpu()`, `sysconf()`
Networking	`socket()`, `setsockopt()`, `bind()`, `listen()`, `accept()`, `recv()`, `send()`, `close()`
File and directory access	`fopen()`, `fread()`, `fgets()`, `opendir()`, `readdir()` on `/proc` and static web files

4.4 POSIX Threads / Synchronization

A design decision of this project is to achieve concurrency with **processes rather than POSIX threads**. The server does not create `pthread` threads; each alarm is a separate process. This choice is deliberate: signals such as `SIGALRM` and the `alarm()` timer are per-process, so one process per alarm gives every alarm its own independent timer and its own PID for `kill()`, and it makes the process states visible to the operating system tools.

Synchronization between the concurrent processes is nevertheless essential and is achieved without threads using the following mechanisms.

- **Signal masking:** `sigprocmask()` blocks the three signals while the worker prepares, and `sigsuspend()` atomically unblocks them and sleeps, which removes the lost-wake-up race.
- **Async-signal-safe flags:** the handlers only set `volatile sig_atomic_t` flags (`g_fired`, `g_cancelled`, `g_dismissed`) and store a timestamp; all real work is done in normal code after the wake-up.
- **Database-level locking:** SQLite in WAL mode lets the server and workers read and write the same file; `sqlite3_busy_timeout()` makes a process wait up to 3 seconds instead of failing with "database is locked".
- **Conditional state changes:** each status update is an SQL statement that only succeeds

from the expected previous state (for example `UPDATE ... WHERE status = 'active'`), so concurrent changes cannot overwrite each other.

4.5 Linux Commands and Utilities

Table 4.4: Commands used during development and demonstration

Command	Purpose
`make`, `make clean`, `make run`	Build, rebuild from scratch, build and start
`./alarm_server [port] [db_path] [public_dir]`	Start the server (defaults: 8080, `db/alarms.db`, `public/`)
`ps --forest -o pid,ppid,stat,cmd -C alarm_server`	Show the parent and child alarm processes
`pstree -p <server_pid>`	Tree view of the server and its children
`kill -0 <pid>`	Test whether a process exists (the same test used by orphan clean-up)
`curl -X POST ...`	Create, cancel and dismiss alarms from the terminal
`cat /proc/<pid>/status`	Inspect process state and context-switch counters
`notify-send`	Raise a desktop notification (invoked by the worker)
`sudo apt install ...`	Install packages

4.6 Other Tools / Libraries Used

Table 4.5: Other libraries and tools

Tool / library	Role
SQLite 3	Embedded SQL database stored in the single file `db/alarms.db`; WAL journal mode; tables `alarms` and `events`.
libnotify / `notify-send`	Delivers the desktop notification containing the PID, parent PID, alarm ID, CPU core and latency.
HTML5 and CSS3	`public/index.html` and `public/style.css` define the page layout, the scheduler panels, the ring banner and the toast messages.
JavaScript (browser)	`public/app.js` uses the Fetch API to call the REST endpoints and renders the alarm list, scheduler animation, process table and event log.
Web Audio API	Generates the loud, repeating beep pattern that plays until the alarm is stopped.
Headless Chromium	Used only to capture the screenshots shown in Chapter 7.

CHAPTER -5 SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The system has three cooperating parts that all run on one Linux machine: the **browser front end**, the **C server process** and the **per-alarm worker processes**. They share a SQLite database, and all of them rely on kernel services for timers, signals, scheduling and process information.

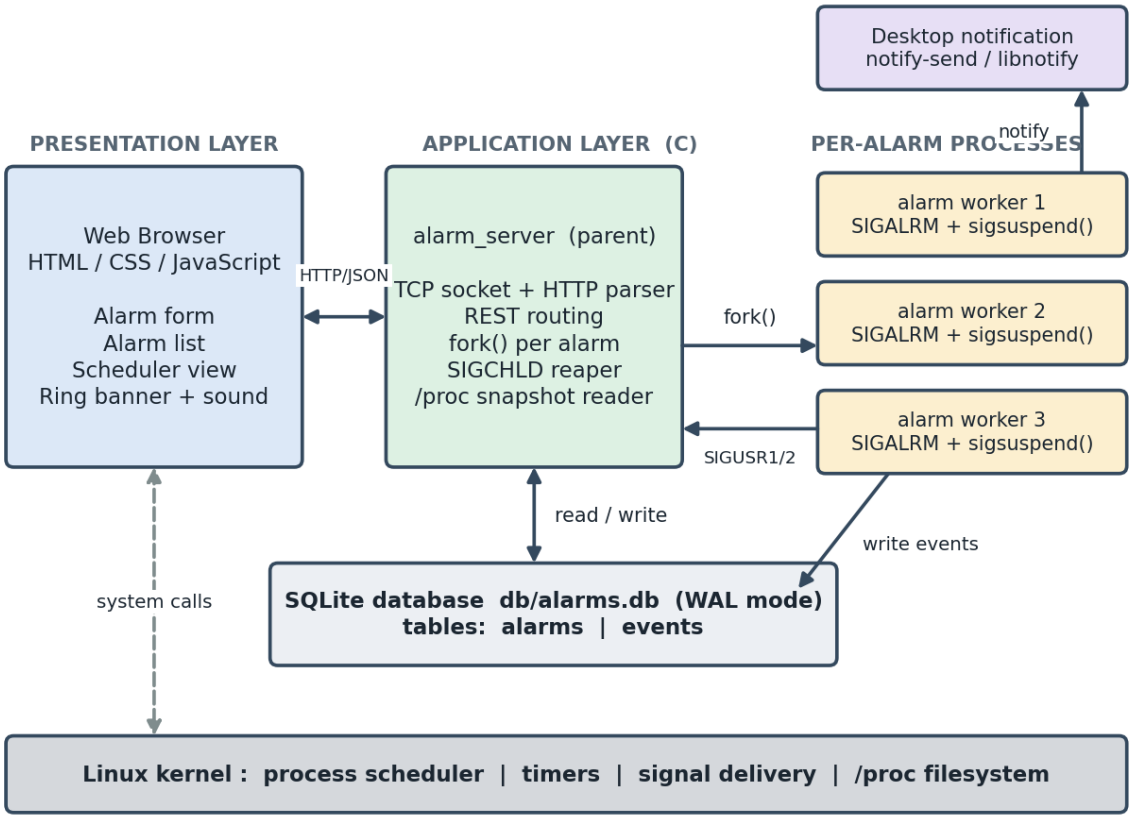


Figure 5.1: Overall system architecture

Table 5.1: Architectural layers

Layer	Component	Responsibility
Presentation	Browser: HTML, CSS, JavaScript	Alarm form, alarm list, scheduler view, process table, event log, ring banner and sound
Application	`alarm_server` (parent process)	Accept TCP connections, parse HTTP, route REST calls, serve static files, fork workers, send control signals, reap children
Execution	Alarm workers (child processes)	Own one alarm each: arm the timer, sleep, handle signals, notify, record events
Data	SQLite file `db/alarms.db`	Persist alarms and scheduler events; act as shared memory between processes
Kernel	Linux	Scheduler, timers, signal delivery, `/proc`, sockets

5.2 System Workflow

The end-to-end workflow of one alarm is as follows.

1. **Request.** The user enters a label and a delay (or a clock time) and presses *Set alarm*. The JavaScript converts a clock time into a delay in seconds and sends `POST /api/alarms` with a JSON body such as `{"label":"Wake up","delay_seconds":6}`.
2. **Validation and storage.** The server extracts the fields, rejects a delay smaller than 1 second with HTTP 400, and inserts a row with status `active`.
3. **Process creation.** The server records the current time and calls `fork()`. In the child, the listening socket and the client socket are closed and `run_alarm_worker()` starts. In the parent, the child's PID is stored with `db_set_pid()` and returned to the browser.
4. **Waiting.** The worker logs the events NEW, START and WAIT, installs the handlers, blocks the signals, calls `alarm(delay)` and sleeps in `sigsuspend()`.
5. **Alarm event.** After the delay the kernel sends `SIGALRM`. The worker wakes, computes timer latency and run-queue wait, logs READY and RUNNING, sets the status to `ringing`, prints to the console and calls `notify-send`.
6. **Ringing.** The browser sees the status `ringing` (it polls every second), shows the red banner, plays the alarm sound and flashes the page title. The worker waits again in `sigsuspend()`.
7. **Stop or cancel.** *Stop alarm* calls `POST /api/alarms/<id>/dismiss`, and the server sends `SIGUSR2`. A *Cancel* on a waiting alarm calls `.../cancel`, and the server sends `SIGUSR1`.
8. **Exit.** The worker updates the status to `triggered` or `cancelled`, logs EXIT, closes the database and calls `exit(0)`. The `SIGCHLD` handler in the server reaps it.

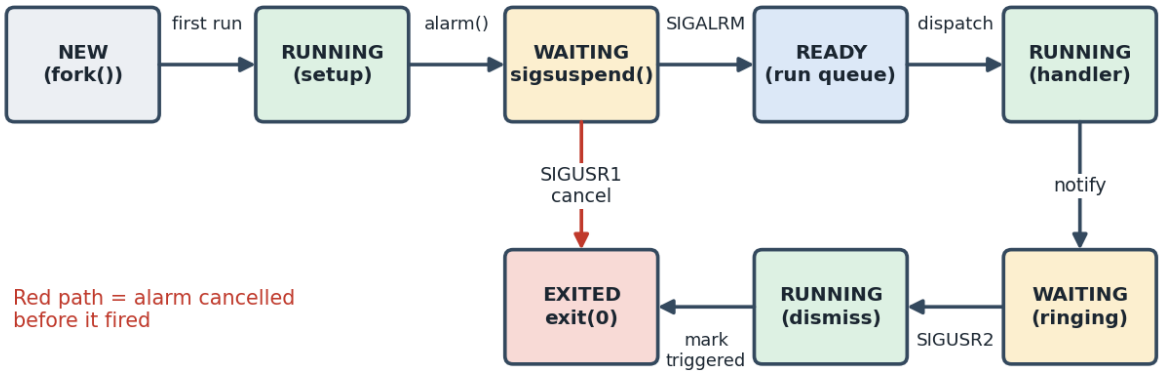


Figure 5.2: Life-cycle states of an alarm worker process

5.3 Flowchart / Data Flow

Figure 5.3 shows the flowchart of the alarm worker, which is the heart of the system. The decision after `sigsuspend()` separates the cancel path from the normal ringing path.

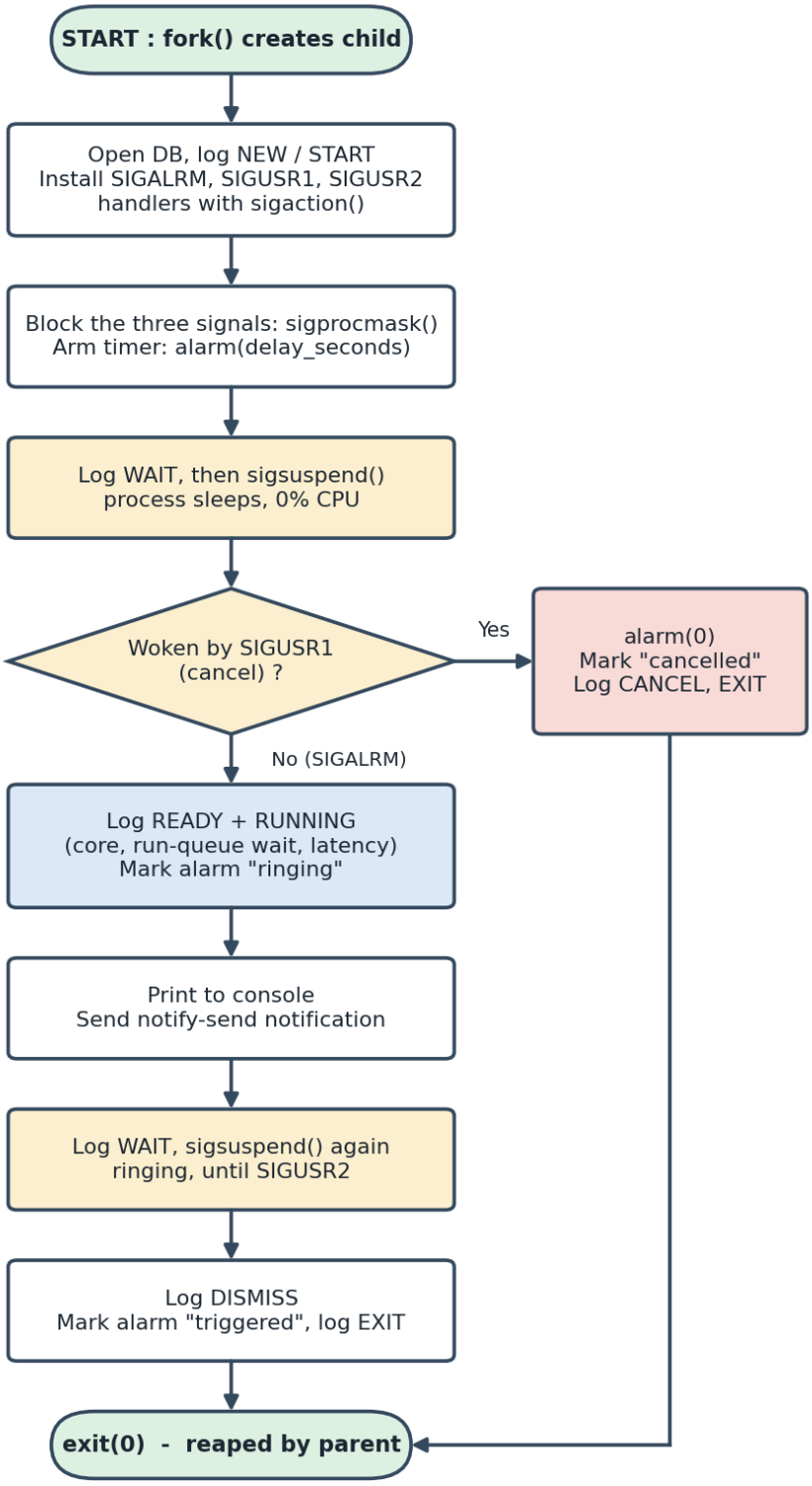


Figure 5.3: Flowchart of the alarm worker process

Figure 5.4 is the data-flow diagram. The user interacts only with the server (process 1.0). The server creates alarm workers (process 2.0 forks process 3.0), workers write alarm status to table D1 and scheduler events to table D2, and the scheduler monitor (process 4.0) combines the

events and live `/proc` data into the snapshot shown in the browser.

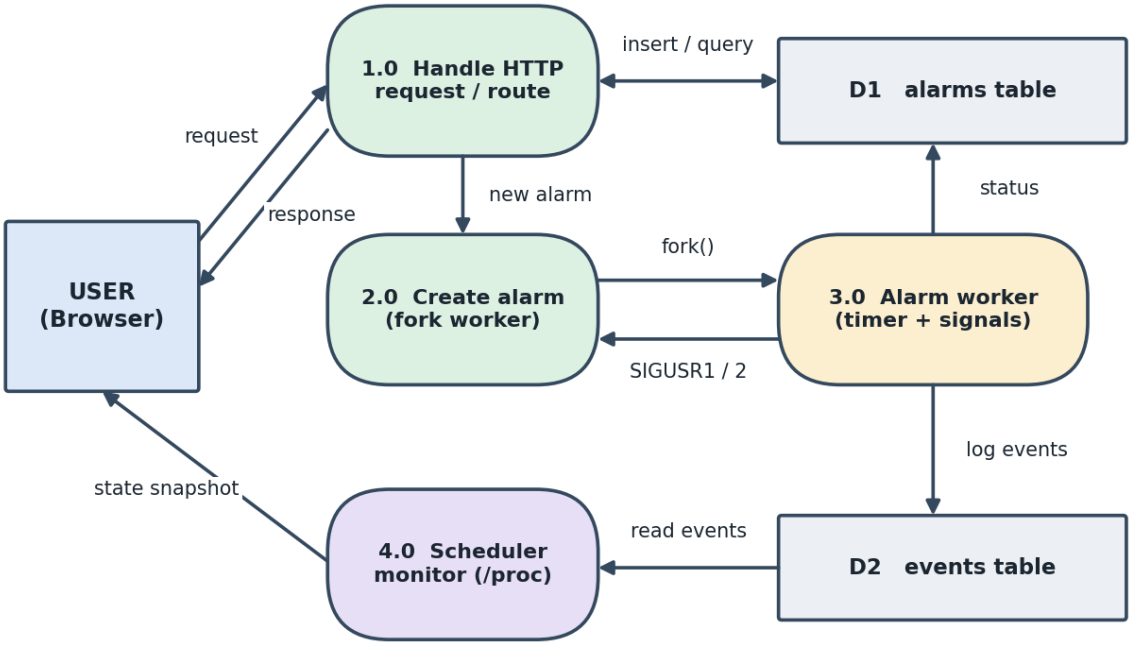


Figure 5.4: Data-flow diagram of the system

5.4 Module Description

Table 5.2: Module description

Module	Key functions	Description
`server.c`	`main()`, `route()`, `handle_post_alarms()`, `handle_cancel_alarm()`, `handle_dismiss_alarm()`, `sigchld_handler()`	Creates the listening socket, installs the `SIGCHLD` reaper, accepts connections one at a time, routes requests, forks workers and sends `SIGUSR1` / `SIGUSR2`.
`alarm_worker.c`	`run_alarm_worker()`, `on_sigalrm()`, `send_linux_notification()`, `runqueue_wait_ns()`	Runs inside the forked child. Implements the timer, the two `sigsuspend()` waits, the notification and the logging of scheduler events.
`db.c`	`db_open()`, `db_insert_alarm()`, `db_mark_ringing()`, `db_alarms_to_json()`, `db_add_event()`, `db_cleanup_orphans()`	Creates the tables, enables WAL mode, performs all SQL operations with prepared statements, and builds JSON for the API.
`http.c`	`http_read_request()`, `http_send_response()`, `http_send_file()`, `json_get_string()`, `json_get_int()`	Reads one HTTP request from a socket, writes responses, serves static files and extracts fields from the fixed JSON bodies.

Module	Key functions	Description
`sysinfo.c`	`sysinfo_json()`, `read_stat()`, `ctx_switches()`	Builds the JSON snapshot of per-core usage, load average, process counts and per-process state from `/proc`.
`public/index.html`, `style.css`	-	Page structure and styling of the form, alarm list, scheduler panels, process table, event log and banner.
`public/app.js`	`refresh()`, `pollEvents()`, `pollSystem()`, `startRing()`	Front-end logic: form handling, polling, scheduler animation, alarm sound, toasts.

5.5 Input and Output Design

Inputs

Table 5.3: Inputs and validation

Input	Source	Validation
Label	Text box (up to 60 characters)	Defaults to "Alarm" when empty; stored as text and escaped before JSON output
Delay (h, m, s)	Number boxes in *Delay* mode	Converted to seconds; must total at least 1 second
Clock time	Time picker in *Clock time* mode	If the time has already passed today, tomorrow is used
Alarm ID in URL	`/api/alarms/<id>/cancel` or `/dismiss`	Alarm must exist and be in the correct status (404 or 400 otherwise)

REST interface

Table 5.4: REST API endpoints

Method and path	Purpose	Typical response
`GET /api/alarms`	List all alarms, newest first	JSON array of alarm objects
`POST /api/alarms`	Create an alarm	`{"id":1,"pid":610,"trigger_at":...,"status":"active"}`
`POST /api/alarms/<id>/cancel`	Cancel an active alarm (`SIGUSR1`)	`{"ok":true}`
`POST /api/alarms/<id>/dismiss`	Stop a ringing alarm (`SIGUSR2`)	`{"ok":true}`
`POST /api/alarms/clear`	Delete finished alarms and their events	`{"ok":true}`
`GET /api/events?after=N`	Scheduler events with id greater than N	`{"last":N,"events":[...]}`
`GET /api/system`	Live CPU and process snapshot from `/proc`	JSON with cores, load, counts, processes
`GET /` and other paths	Static files from `public/`	HTML, CSS or JavaScript

Database design

Table 5.5: Database tables

Table	Columns	Notes
`alarms`	`id` (PK, autoincrement), `label`, `delay_seconds`, `created_at`, `trigger_at`, `status`, `pid`	Status lifecycle: `active` to `ringing` to `triggered`, or `active` to `cancelled`
`events`	`id` (PK), `alarm_id`, `pid`, `ts_us`, `type`, `state`, `cpu`, `detail`	One row for every step a worker takes; `ts_us` is a Unix time in microseconds

Outputs

- **Server console:** start-up banner, an `ALARM!` line with PID, CPU core, run-queue wait and latency, and `dismissed` / `cancelled` lines.
- **Desktop notification:** title "ALARM TRIGGERED: label" with PID, parent PID, alarm ID, core and timing, shown through `notify-send`.
- **Web page:** red banner and toast ("Alarm triggered"), repeating alarm sound until stopped, flashing page title, and status badges (active, ringing, done, cancelled).
- **Scheduler view:** chips moving between the New, Wait queue, Ready queue, CPU core and Exited panels, a live `/proc` process table and a timestamped event log.

CHAPTER -6 IMPLEMENTATION

6.1 Module-wise Implementation

Server module (server.c)

`main()` reads optional command-line arguments (port, database path, public directory), ignores `SIGPIPE`, installs the `SIGCHLD` handler with `SA_RESTART`, opens the database, marks orphaned alarms as cancelled, and creates the listening socket with `SO_REUSEADDR`. It then enters an endless `accept()` loop: for each connection it reads one request, calls `route()` and closes the socket. The server handles one request at a time, which is sufficient for the interface and keeps the code focused on the operating-system concepts.

Alarm worker module (alarm_worker.c)

`run_alarm_worker()` runs in the child. Its steps are listed in Section 6.2. Besides the timer logic it measures two scheduling quantities: **timer latency** (time of signal delivery minus the expected time) and **run-queue wait** (difference of the second field of `/proc/self/schedstat` before and after the sleep). The notification is sent by forking again and replacing the grandchild with `notify-send` through `execlp()`.

Database module (db.c)

`db_open()` opens the file, sets a 3-second busy time-out, turns on WAL mode and creates the `alarms` and `events` tables if they do not exist. All statements use prepared statements with bound parameters, which avoids SQL injection from the alarm label. Status changes are made by `mark_status()`, which updates a row only if it currently has the expected status, and reports whether a row was changed.

HTTP module (http.c)

`http_read_request()` receives bytes with `recv()` until the blank line that ends the headers, parses the method and path with `sscanf()`, finds the `Content-Length` header, and reads the body. `http_send_response()` writes the status line and headers (`Content-Type`, `Content-Length`, `Connection: close`) followed by the body. `http_send_file()` loads a static file and chooses a content type from the extension.

System-information module (sysinfo.c)

`sysinfo_json()` reads `/proc/loadavg`, computes per-core usage from two successive readings of `/proc/stat`, walks `/proc` to count processes by state, and adds detailed rows for the server, every active worker and a few other runnable processes.

6.2 Algorithm / Pseudocode

Algorithm 1: Server main loop

```
1.  Parse arguments; ignore SIGPIPE; install SIGCHLD handler
2.  Open database; mark orphaned alarms as cancelled
3.  Create socket; bind; listen
4.  LOOP forever
5.      client <- accept()
6.      IF read_request(client) succeeds THEN route(client, request)
7.      close(client)
```

Algorithm 2: Create an alarm (parent side)

```
1.  label <- json "label"; delay <- json "delay_seconds"
2.  IF delay < 1 THEN reply 400 "delay_seconds must be a positive integer"
3.  id <- INSERT alarm (status = 'active')
4.  pid <- fork()
5.  IF pid == 0 THEN                        (child)
6.      close listening and client sockets
7.      run_alarm_worker(id, delay, label)      -- never returns
8.  ELSE                                   (parent)
9.      UPDATE alarm SET pid = pid
10.     reply 200 {id, pid, trigger_at, status}
```

Algorithm 3: Alarm worker

```
1.  open database; reset SIGCHLD to default
2.  log NEW, START
3.  install handlers: SIGALRM, SIGUSR1, SIGUSR2   (set flags only)
4.  block those three signals (sigprocmask)
5.  expected <- now + delay;  alarm(delay)
6.  log WAIT
7.  WHILE no alarm and no cancel: sigsuspend(original_mask)
8.  IF cancelled THEN alarm(0); status <- cancelled; log CANCEL, EXIT; exit(0)
9.  latency <- signal_time - expected;  log READY, RUNNING
10. status <- ringing;  print line;  notify-send;  log NOTIFY
11. log WAIT (ringing)
12. WHILE not dismissed and not cancelled: sigsuspend(original_mask)
13. log DISMISS;  status <- triggered;  log EXIT;  exit(0)
```

Algorithm 4: Cancel / dismiss (parent side)

```
1.  (pid, status) <- SELECT FROM alarms WHERE id = id
2.  IF not found THEN 404
3.  cancel : require status == 'active'   ->  kill(pid, SIGUSR1)
4.  dismiss: require status == 'ringing'  ->  kill(pid, SIGUSR2)
5.  IF the required status does not match THEN 400; IF kill fails THEN 500
6.  reply {"ok": true}
```

6.3 Core C Implementation

The most important code is the signal set-up and wait in `alarm_worker.c`. The handlers only set flags and take a timestamp, which are async-signal-safe actions.

```
static volatile sig_atomic_t g_fired = 0, g_cancelled = 0, g_dismissed = 0;
static volatile long long g_sig_us = 0;


static void on_sigalrm(int s) { (void)s; g_sig_us = now_us(); g_fired = 1; }
static void on_sigusr1(int s) { (void)s; g_cancelled = 1; }
static void on_sigusr2(int s) { (void)s; g_dismissed = 1; }
```

Listing 6.1: Signal handlers in alarm_worker.c

```
struct sigaction sa;
memset(&sa, 0, sizeof(sa));
sigemptyset(&sa.sa_mask);
sa.sa_handler = on_sigalrm; sigaction(SIGALRM, &sa, NULL);
sa.sa_handler = on_sigusr1; sigaction(SIGUSR1, &sa, NULL);
sa.sa_handler = on_sigusr2; sigaction(SIGUSR2, &sa, NULL);

sigset_t block, orig;
sigemptyset(&block);
sigaddset(&block, SIGALRM);
sigaddset(&block, SIGUSR1);
sigaddset(&block, SIGUSR2);
sigprocmask(SIG_BLOCK, &block, &orig);

long long expected_us = now_us() + (long long)delay_seconds * 1000000LL;
alarm((unsigned int)delay_seconds);

while (!g_fired && !g_cancelled) sigsuspend(&orig);
```

Listing 6.2: Installing handlers, blocking signals, arming the timer and sleeping

Because the signals are blocked before `alarm()` is called and `sigsuspend()` unblocks them atomically while sleeping, a signal can never be lost between the flag test and the sleep. The `while` loop also protects against spurious wake-ups by other signals.

```
pid_t pid = fork();
if (pid < 0) {
    http_send_json(fd, 500, "{\"error\":\"fork failed\"}");
    return;
}

if (pid == 0) {
    /* Child: this process's whole job in life is this one alarm. */
    close(g_listen_fd);
    close(fd);
    run_alarm_worker(g_db_path, id, (int)delay_seconds,
                     label, fork_us);          /* never returns */
}
/* Parent: record which pid owns this alarm (needed for cancellation). */
db_set_pid(g_db, id, pid);
```

Listing 6.3: Creating the worker with fork() in server.c

```
static void sigchld_handler(int sig) {
    (void)sig;
    int saved_errno = errno;
    while (waitpid(-1, NULL, WNOHANG) > 0) { }
    errno = saved_errno;
}
```

Listing 6.4: Reaping finished workers (prevents zombies)

```
static int mark_status(sqlite3 *db, long long id,
                       const char *from, const char *new_status) {
    const char *sql =
        "UPDATE alarms SET status = ? WHERE id = ? AND status = ?;";
    ...
    return (rc == SQLITE_DONE && changed > 0) ? 0 : -1;
}
int db_mark_ringing(sqlite3 *db, long long id)
    { return mark_status(db, id, "active", "ringing"); }
int db_mark_triggered(sqlite3 *db, long long id)
    { return mark_status(db, id, "ringing", "triggered"); }
int db_mark_cancelled(sqlite3 *db, long long id)
    { return mark_status(db, id, "active", "cancelled"); }
```

Listing 6.5: Conditional status transitions in db.c

6.4 Linux System Calls / APIs Used

Table 6.1: System calls and APIs used, with location and purpose

Call / API	File	Purpose
`fork()`	`server.c`, `alarm_worker.c`	One child per alarm; a second fork inside the worker to run `notify-send`
`execlp()`	`alarm_worker.c`	Replace the notification child with the `notify-send` program
`waitpid()`	`server.c`, `alarm_worker.c`	Reap alarm workers in the server; wait for `notify-send` in the worker
`exit()`, `_exit()`	`alarm_worker.c`	Terminate the worker normally (0) or the failed `exec` child (127)
`sigaction()`	`server.c`, `alarm_worker.c`	Install `SIGCHLD`, `SIGALRM`, `SIGUSR1`, `SIGUSR2` handlers
`sigprocmask()` and `sigsuspend()`	`alarm_worker.c`	Block signals, then sleep atomically until one arrives
`alarm()`	`alarm_worker.c`	Arm the kernel timer; `alarm(0)` cancels it
`kill()`	`server.c`, `db.c`	Send `SIGUSR1` / `SIGUSR2`; `kill(pid, 0)` tests that a process exists
`clock_gettime()`	`alarm_worker.c`, `db.c`, `sysinfo.c`	Microsecond timestamps for latency and events
`sched_getcpu()`	`alarm_worker.c`	Report which CPU core the worker is running on
`socket()`, `bind()`, `listen()`, `accept()`	`server.c`	Create the TCP server
`recv()`, `send()`	`http.c`	Read requests and write responses
`getpid()`, `getppid()`, `getuid()`	`alarm_worker.c`	Identify the process in messages and locate the user's D-Bus session
`/proc` files	`sysinfo.c`, `alarm_worker.c`	`stat`, `status`, `wchan`, `loadavg`, `schedstat`
SQLite C API	`db.c`	`sqlite3_open`, `prepare_v2`, `bind_*`, `step`, `finalize`, `busy_timeout`

6.5 Process / Thread Implementation

Every alarm has its own process. Figure 6.1 summarises the resulting process tree: one long-lived server and one short-lived child for each alarm that is active or ringing. A child exists only from the moment the alarm is created until it is cancelled or stopped.

```
alarm_server (server, pid 607)
 |-- alarm_server (worker for alarm 1, pid 610)  -- sleeping in sigsuspend()
 |-- alarm_server (worker for alarm 2, pid 622)  -- sleeping in sigsuspend()
 `-- alarm_server (worker for alarm 3, pid 631)  -- ringing (SIGUSR2 wait)
```

Figure 6.1: Process tree during operation (illustrative)

- **No threads:** the server is single-threaded and uses `accept()` in a loop; concurrency

comes from the worker processes (see Section 4.4).

- **Child set-up after fork:** the child closes the listening socket and the client socket it does not need, and opens its own SQLite connection, because a database handle must not be shared across `fork()`.
- **Signal disposition:** the child resets `SIGCHLD` to the default action because the inherited reaper would interfere with the `waitpid()` it uses for the `notify-send` child.
- **Process-state mapping:** NEW is the moment after `fork()`, RUNNING is the first execution and the handler execution, WAITING is time spent in `sigsuspend()`, READY is the time between `SIGALRM` and dispatch on a CPU, and EXITED is after `exit(0)`.
- **Termination:** a worker always ends through `exit(0)`; the parent's `SIGCHLD` handler removes it from the process table.

6.6 IPC / Synchronization Implementation

Signals as command channel

The server controls a worker only through signals: `SIGUSR1` means cancel and `SIGUSR2` means dismiss. The server first checks the alarm's status in the database so that the right signal is sent only in the right state.

SQLite as shared data channel

The server and workers exchange data through the database file. The server writes the alarm row and the PID; the worker writes status changes and events; the browser reads the combined result. WAL mode allows readers and a writer to work at the same time, and the 3-second busy time-out resolves short write conflicts.

Table 6.2: Alarm status transitions (compare-and-set)

Transition	Performed by	Allowed only when status is	Trigger
`active` to `ringing`	Worker	`active`	`SIGALRM` received
`ringing` to `triggered`	Worker	`ringing`	`SIGUSR2` (dismiss)
`active` to `cancelled`	Worker	`active`	`SIGUSR1` (cancel)
`active` / `ringing` to `cancelled`	Server at start-up	worker PID no longer exists	`kill(pid, 0)` fails with `ESRCH`

Avoiding race conditions

- The signals are blocked before the timer is armed, and `sigsuspend()` waits with the original mask, so no wake-up can be lost.
- Handlers do only async-signal-safe work (flag assignment and `clock_gettime()`); printing and database access happen outside the handler.
- Each status update is a single SQL statement that checks the current status, so two actors cannot move an alarm to conflicting states.
- The server's `SIGCHLD` handler saves and restores `errno` so that it does not disturb an interrupted system call.

6.7 Memory Management Implementation

The program uses small fixed-size buffers on the stack and heap, and takes care that every copy is bounded.

Table 6.3: Memory handling

Area	Technique	Reason
JSON responses	`malloc(65536)` per request in the handlers, released with `free()` before returning	Large buffer for alarm lists and event batches without using a very large stack frame
HTTP request buffer	One `static char[65536]` inside `http_read_request()`	Safe because the server handles one request at a time; avoids repeated allocation
Request structure	Fixed arrays: method 8, path 512, body 8192 bytes	Body and path are truncated to the array size, preventing buffer overflow
String formatting	`snprintf()` with the buffer size everywhere	Prevents overruns when building messages and JSON
JSON escaping	`json_escape_append()` checks remaining space before each write	Labels containing quotes or newlines cannot break the JSON or overflow the row buffer
Static file serving	`malloc(size)` for the file, `free()` after sending	Files are read whole and sent with correct `Content-Length`
After `fork()`	Copy-on-write address space; child opens its own database handle	Each worker has an independent memory image; pages are shared until modified
Path safety	`path_is_safe()` rejects any path containing `..`	Stops requests from escaping the `public/` directory

Each worker is a copy of the server image, but because Linux uses copy-on-write the extra memory per sleeping worker is small. The program also closes every file it opens, finalises each prepared statement, and closes the database before a worker exits.

6.8 Optional User Interface Implementation

The user interface is a single-page web application served by the same C binary from the `public/` directory. It uses plain HTML, CSS and JavaScript with no external framework.

Layout (index.html and style.css)

- **Header:** product name and a live digital clock updated every second.
- **New alarm card:** label field, *Delay* / *Clock time* toggle, hour-minute-second inputs or a time picker, *Set alarm* button and *Test sound* button.
- **Alarms card:** list of alarms with a status badge, the PID, a countdown, and a *Cancel* or *Stop* button; a *Clear finished* button.
- **CPU scheduler card:** statistics chips (load, process counts), five panels (New, Wait queue, Ready queue, CPU cores, Exited), a slow-motion switch, the live process table and the event log.
- **Ring banner and toasts:** a red banner at the top with a *Stop alarm* button, and pop-up toast messages.

Behaviour (app.js)

Table 6.4: Front-end polling and behaviour

Function	Interval	Description
`refresh()`	Every 1 s	Fetches `/api/alarms`, redraws the list and countdowns, starts or stops the ring banner and sound
`pollEvents()`	Every 400 ms	Fetches new rows from `/api/events?after=N` and queues them for playback
Event player	Every 100 ms	Takes events from the queue and moves chips between panels; slow motion lengthens each step so it can be followed
`pollSystem()`	Every 600 ms	Fetches `/api/system` and updates core usage, statistics and the live process table
`startRing()`	Every 1.6 s while ringing	Plays a four-beep pattern with the Web Audio API until the alarm is stopped

The ring banner appears when any alarm has the status `ringing`; the page title flashes, and **Stop alarm** sends `POST /api/alarms/<id>/dismiss` for every ringing alarm. Chips in the scheduler panels are driven by the events that the worker processes logged themselves, so the picture shows what the operating system actually did.

CHAPTER -7 EXECUTION AND DEMONSTRATION

All results in this chapter were produced by building and running the submitted source code on Ubuntu 24.04 (single virtual CPU) with SQLite 3 and GCC. The outputs and screenshots are real captures, not mock-ups.

7.1 Compilation in Ubuntu

```
sudo apt install -y build-essential libsqlite3-dev libnotify-bin
make && ./alarm_server     # open http://localhost:8080
```

Listing 7.1: Installing prerequisites, building and starting the server

The commands above install the required packages, build the project, and start alarm_server on the default port 8080. Then open http://localhost:8080 in a browser.

Listing 7.1: Installing prerequisites and building

The build output shows each source file compiled with the warning options and the final link step. No warnings or errors were reported.

```
mkdir -p build
gcc -Wall -Wextra -std=gnu11 -Iinclude -g -c src/alarm_worker.c \
    -o build/alarm_worker.o
gcc -Wall -Wextra -std=gnu11 -Iinclude -g -c src/db.c -o build/db.o
gcc -Wall -Wextra -std=gnu11 -Iinclude -g -c src/http.c -o build/http.o
gcc -Wall -Wextra -std=gnu11 -Iinclude -g -c src/server.c -o build/server.o
gcc -Wall -Wextra -std=gnu11 -Iinclude -g -c src/sysinfo.c -o build/sysinfo.o
gcc build/alarm_worker.o build/db.o build/http.o build/server.o \
    build/sysinfo.o -o alarm_server -lsqlite3
```

Listing 7.2: Build output (successful, no warnings)

To rebuild from scratch, use `make clean && make`. The result is the executable `alarm_server` (about 94 KB).

7.2 Execution Commands

```
make && ./alarm_server     # open http://localhost:8080
./alarm_server 9000            # custom port
./alarm_server <port> <db_path> <public_dir>
make run                   # builds, creates db/ and starts the server
```

Listing 7.3: Starting the server

After the start-up banner appears, open http://localhost:8080 in a browser. In a second terminal the process tree can be watched while alarms are created.

```
ps --forest -o pid,ppid,stat,cmd -C alarm_server
pstree -p <server_pid>

# the same operations without the browser
curl -X POST localhost:8080/api/alarms -H 'Content-Type: application/json' \
     -d '{"label":"Wake up","delay_seconds":6}'
curl localhost:8080/api/alarms
curl -X POST localhost:8080/api/alarms/1/dismiss
curl -X POST localhost:8080/api/alarms/2/cancel
```

Listing 7.4: Observing and testing from the terminal

7.3 Sample Inputs and Outputs

Test 1: Create an alarm for 6 seconds

```
$ ./alarm_server
Alarm Clock System server listening on http://localhost:8080
Database: db/alarms.db | Static files: public

$ curl -X POST localhost:8080/api/alarms -H 'Content-Type: application/json' \
       -d '{"label":"Wake up","delay_seconds":6}'
{"id":1,"pid":610,"trigger_at":1791189248,"status":"active"}

$ ps --forest -o pid,ppid,stat,cmd -C alarm_server
  PID  PPID STAT CMD
  607   605 Ss   ./alarm_server
  610   607 R     \_ ./alarm_server
```

Listing 7.5: Alarm creation and the new child process

The response contains the alarm ID and the PID of the new child (610), whose parent PID is the server (607). The `ps --forest` output confirms that a child process now exists for the alarm.

Test 2: The alarm fires

```
$ curl localhost:8080/api/alarms
[{"id":1,"label":"Wake up","delay_seconds":6,"created_at":1791189242,
  "trigger_at":1791189248,"status":"ringing","pid":610}]

Server console:
[alarm-worker pid=610] ALARM! "Wake up" (id=1) cpu=0
    runqueue_wait=10us latency=153us
```

Listing 7.6: Alarm status and console message after the timer expires

After 6 seconds the status is `ringing`. The worker measured a timer latency of 153 microseconds and a run-queue wait of 10 microseconds on CPU 0.

Test 3: Scheduler event log for the alarm

```
id  pid  type     state    cpu  detail
1   610  NEW      NEW      -1   fork() created this process (parent pid 607)
2   610  START    RUNNING   0   First time on a CPU: installing SIGALRM/
                                 SIGUSR1/SIGUSR2 handlers, arming alarm()
3   610  WAIT     WAITING   0   sigsuspend(): blocked, removed from the run
                                 queue, using 0% CPU until SIGALRM in 6s
4   610  READY    READY     0   Timer expired, SIGALRM sent: kernel moved the
                                 process from the wait queue to the run queue
                                 (timer scheduled for 1791189248)
5   610  RUNNING  RUNNING   0   Scheduler dispatched it on CPU 0 after 10 us
                                 in the run queue; SIGALRM handler ran
                                 (timer latency 153 us)
6   610  NOTIFY   RUNNING   0   notify-send unavailable (install
                                 libnotify-bin); notified via terminal and
                                 web UI only
7   610  WAIT     WAITING   0   Ringing: blocked in sigsuspend() again,
                                 waiting for dismissal (SIGUSR2)
```

Listing 7.7: Events logged by the worker (output of /api/events)

The sequence NEW, RUNNING, WAITING, READY, RUNNING, WAITING reproduces the textbook process-state model with real timestamps. In the test environment there was no desktop session, so event 6 shows the documented fall-back: the notification was delivered through the

terminal and web page. On an Ubuntu desktop the same step raises a `notify-send` pop-up.

Test 4: Dismiss and cancel

```
$ curl -X POST localhost:8080/api/alarms/1/dismiss
{"ok":true}
$ curl -X POST localhost:8080/api/alarms -H 'Content-Type: application/json' \
       -d '{"label":"Meeting","delay_seconds":60}'
{"id":2,"pid":622,"trigger_at":1791189310,"status":"active"}
$ curl -X POST localhost:8080/api/alarms/2/cancel
{"ok":true}
$ curl localhost:8080/api/alarms
[{"id":2,"label":"Meeting",...,"status":"cancelled","pid":622},
 {"id":1,"label":"Wake up",...,"status":"triggered","pid":610}]

Server console:
[alarm-worker pid=610] alarm "Wake up" (id=1) dismissed.
[alarm-worker pid=622] alarm "Meeting" (id=2) cancelled.
```

Listing 7.8: Dismissing a ringing alarm and cancelling a waiting one

Test 5: Invalid requests

```
$ curl -X POST localhost:8080/api/alarms -H 'Content-Type: application/json' \
       -d '{"label":"x","delay_seconds":0}'
{"error":"delay_seconds must be a positive integer"}

$ curl -X POST localhost:8080/api/alarms/2/cancel   # already cancelled
{"error":"alarm is not active"}
```

Listing 7.9: Input validation and state checking

Table 7.1: Summary of test cases

No.	Test	Input	Expected result	Observed result
1	Create alarm	label "Wake up", 6 s	Row created, child process forked, status `active`	Pass: PID 610 returned, child visible in `ps`
2	Timer fires	wait 6 s	Status `ringing`, console message	Pass: latency 153 us, run-queue 10 us
3	Dismiss ringing alarm	`POST .../1/dismiss`	Status `triggered`, worker exits	Pass; no zombie remained
4	Cancel waiting alarm	`POST .../2/cancel`	Status `cancelled`, worker exits	Pass
5	Zero delay	`delay_seconds = 0`	HTTP 400 error	Pass
6	Cancel non-active alarm	`POST .../2/cancel`	HTTP 400 "alarm is not active"	Pass
7	Child cleanup	`ps` after tests	Only the server process remains	Pass

7.4 Screenshots of Execution

Figure 7.1: Alarm Clock System home page and CPU scheduler

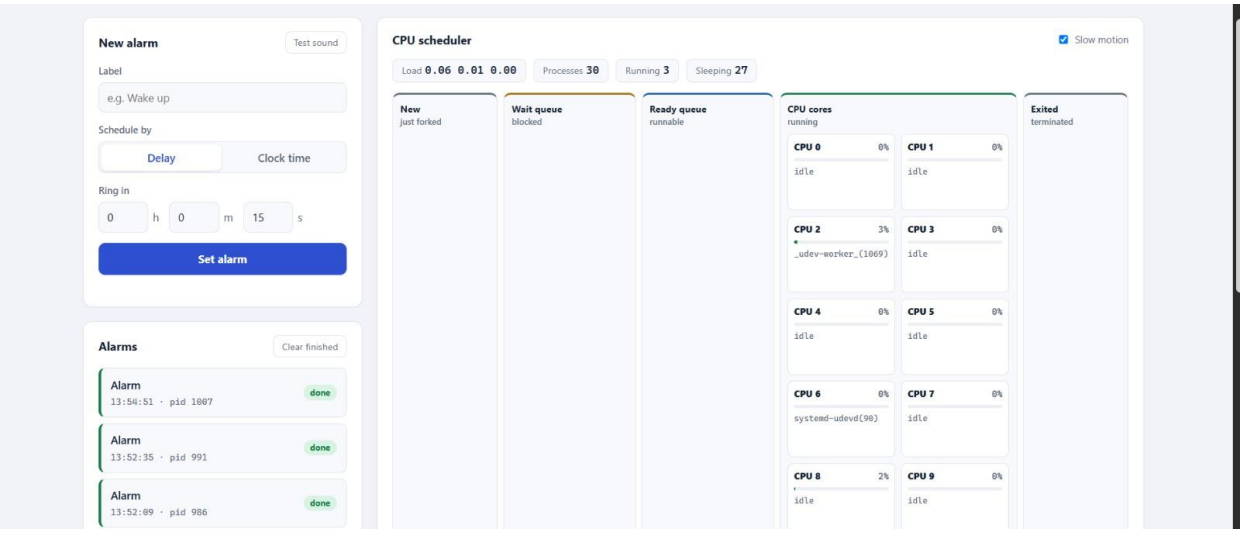


Figure 7.2: An alarm created and waiting in the Wait queue

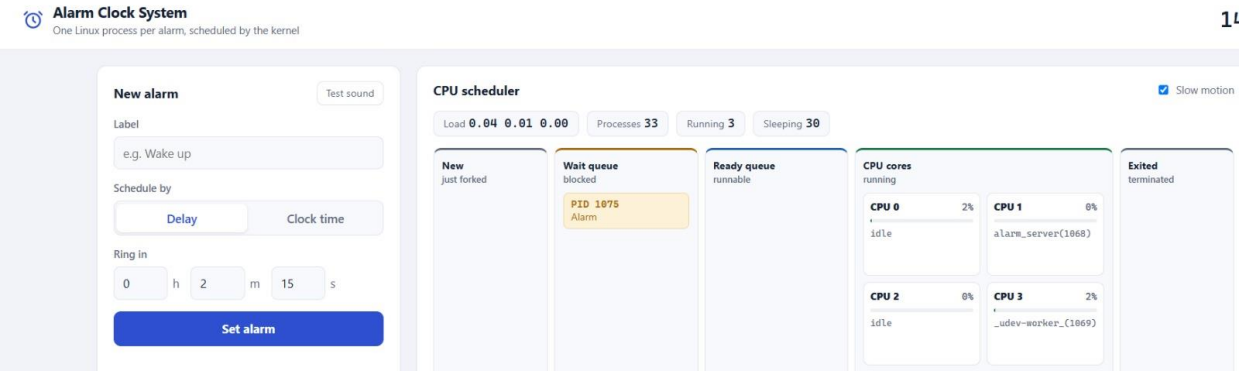


Figure 7.3: Alarm triggered with red notification banner and CPU activity

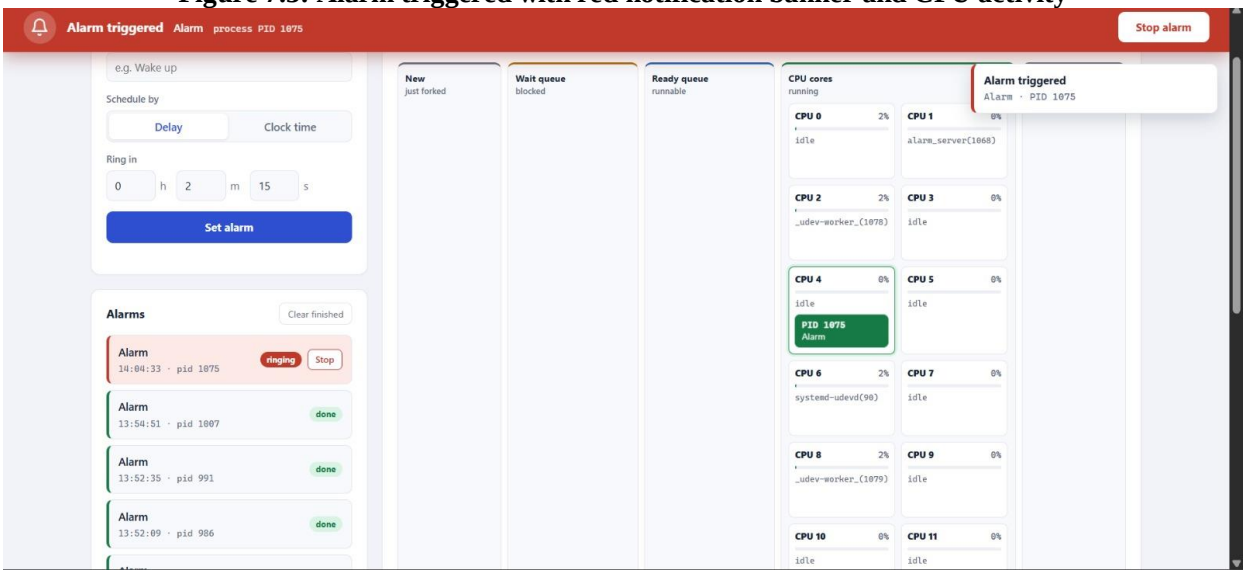


Figure 7.4: Live process table and event log showing alarm process states

Chips are real alarm processes. Moves come from events each worker logs itself (real core and timestamps); slow motion only stretches the playback, the log shows true timings. The process table below is live, so it may already show the next state.

Processes live from /proc

PID	Process	State	Core	CPU time	Context switches
1068	alarm_server (server)	Running	CPU 5	710 ms	4367 voluntary / 8 forced
90	systemd-udevd	Running	CPU 1	7900 ms	24457 voluntary / 148 forced
1076	_udev-worker_	Running	CPU 9	70 ms	144 voluntary / 0 forced
1078	_udev-worker_	Running	CPU 2	1340 ms	1690 voluntary / 9 forced

Event log

```
14:02:18.732  NEW      pid 1075      fork() created this process (parent pid 1068)
14:02:18.756  RUNNING  pid 1075 cpu4 First time on a CPU: installing SIGALRM/SIGUSR1/SIGUSR2 handlers, arming alarm()
14:02:18.764  WAITING  pid 1075 cpu4 sigsuspend(): blocked, removed from the run queue, using 0% CPU until SIGALRM in 135s
14:04:33.764  READY    pid 1075 cpu4 Timer expired, SIGALRM sent: kernel moved the process from the wait queue to the run queue
                                     (timer scheduled for 1791189273)
14:04:33.764  RUNNING  pid 1075 cpu4 Scheduler dispatched it on CPU 4 after 53 us in the run queue; SIGALRM handler ran (timer
                                     latency 273 us)
14:04:33.857  RUNNING  pid 1075 cpu4 notify-send unavailable (install libnotify-bin); notified via terminal and web UI only
14:04:33.864  WAITING  pid 1075 cpu4 Ringing: blocked in sigsuspend() again, waiting for dismissal (SIGUSR2)
14:04:51.621  RUNNING  pid 1075 cpu6 Dismiss signal received: alarm stopped by the user
14:04:51.637  EXITED   pid 1075 cpu6 exit(0): process terminated, parent reaps it via SIGCHLD
```

Figure 7.5: Alarm completed and marked as done

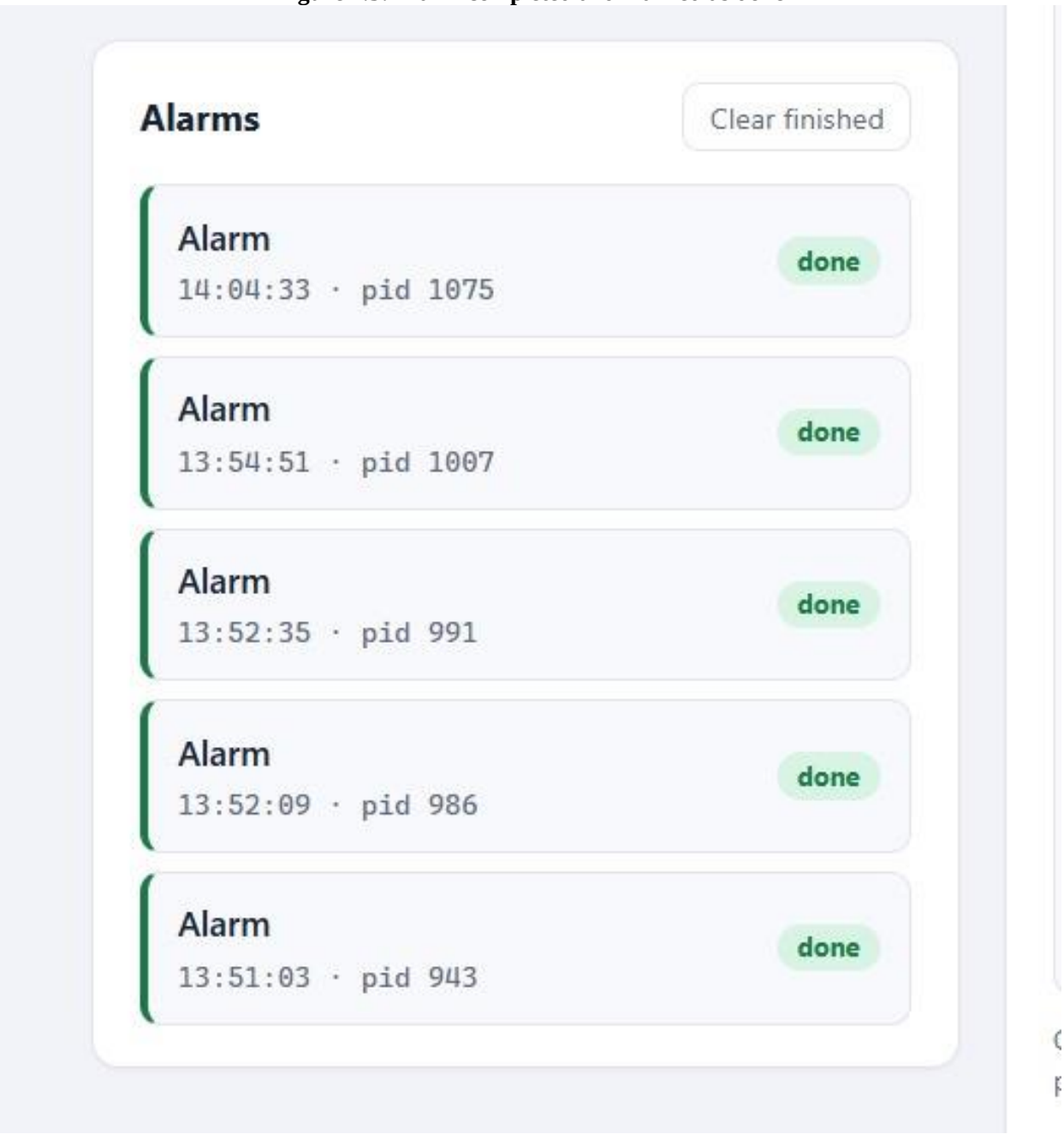


Figure 7.6: Alarm server running successfully in the Linux terminal

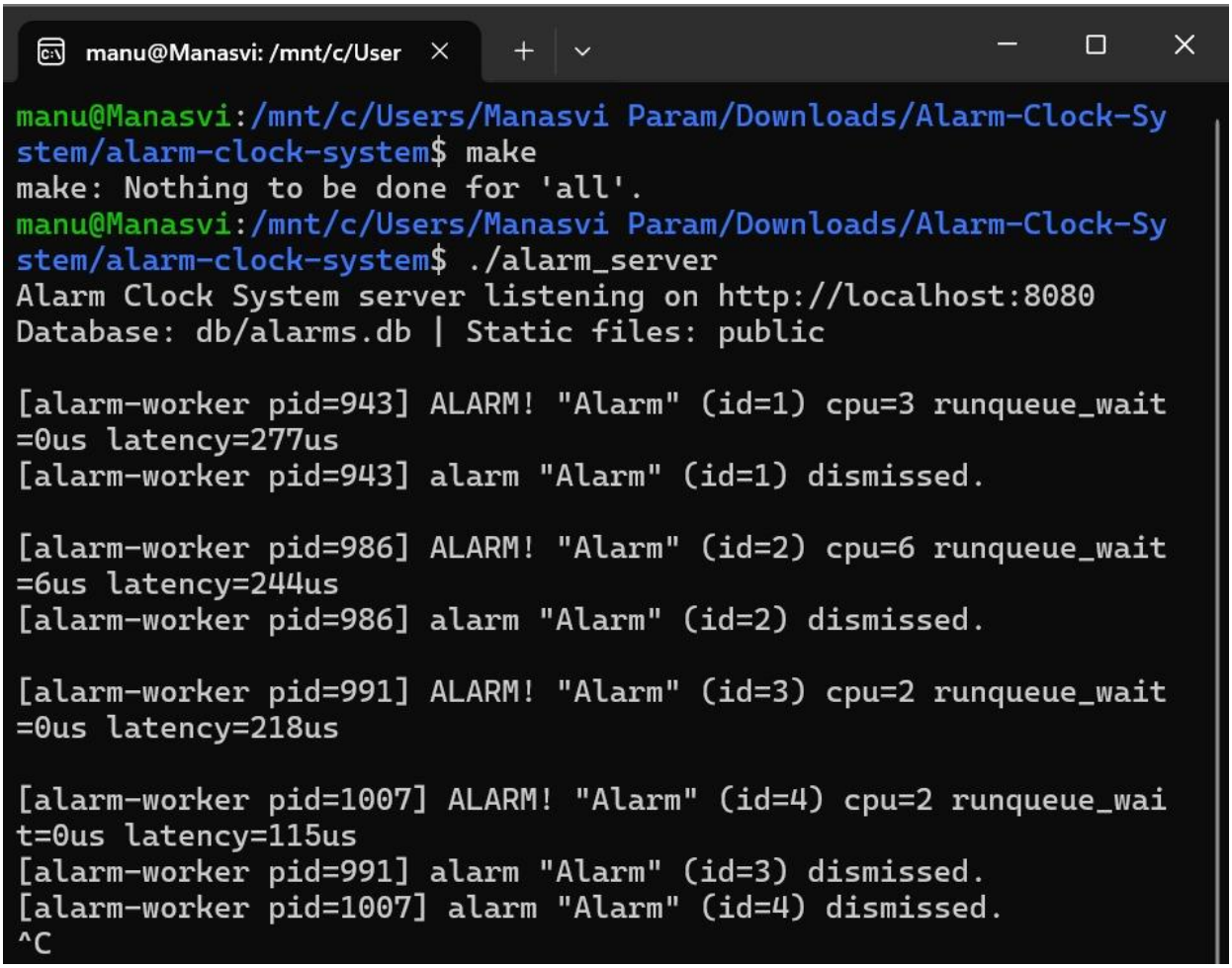

manu@Manasvi: /mnt/c/User
manu@Manasvi:/mnt/c/Users/Manasvi Param/Downloads/Alarm-Clock-System/alarm-clock-system$ make
make: Nothing to be done for 'all'.
manu@Manasvi:/mnt/c/Users/Manasvi Param/Downloads/Alarm-Clock-System/alarm-clock-system$ ./alarm_server
Alarm Clock System server listening on http://localhost:8080
Database: db/alarms.db | Static files: public
[alarm-worker pid=943] ALARM! "Alarm" (id=1) cpu=3 runqueue_wait=0us latency=277us
[alarm-worker pid=943] alarm "Alarm" (id=1) dismissed.
[alarm-worker pid=986] ALARM! "Alarm" (id=2) cpu=6 runqueue_wait=6us latency=244us
[alarm-worker pid=986] alarm "Alarm" (id=2) dismissed.
[alarm-worker pid=991] ALARM! "Alarm" (id=3) cpu=2 runqueue_wait=0us latency=218us
[alarm-worker pid=1007] ALARM! "Alarm" (id=4) cpu=2 runqueue_wait=0us latency=115us
[alarm-worker pid=991] alarm "Alarm" (id=3) dismissed.
[alarm-worker pid=1007] alarm "Alarm" (id=4) dismissed.
^C

Figure 7.7: Clock-time alarm input and cancelled alarm status

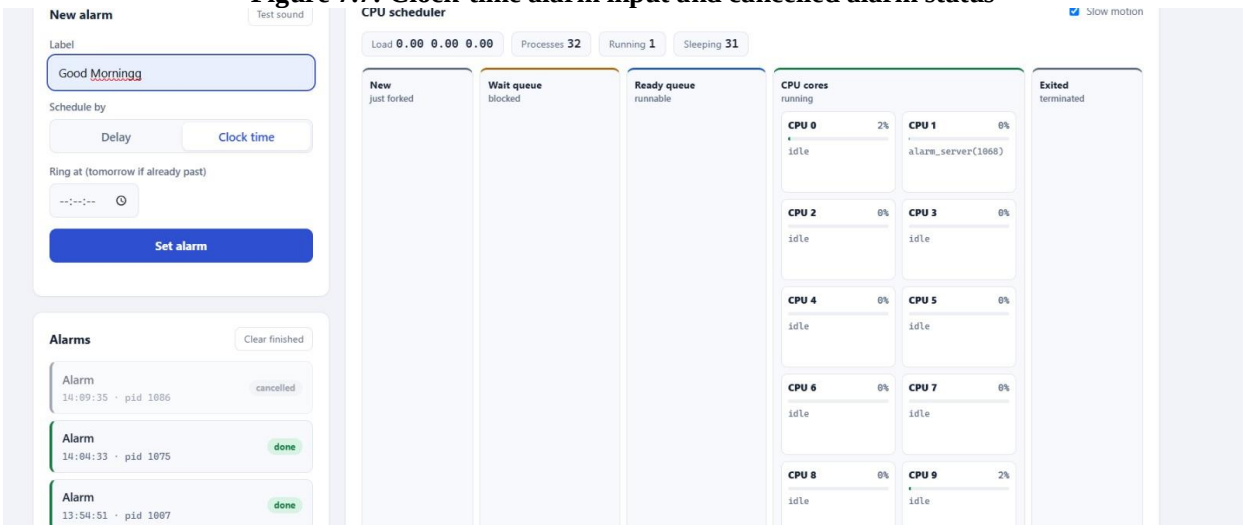
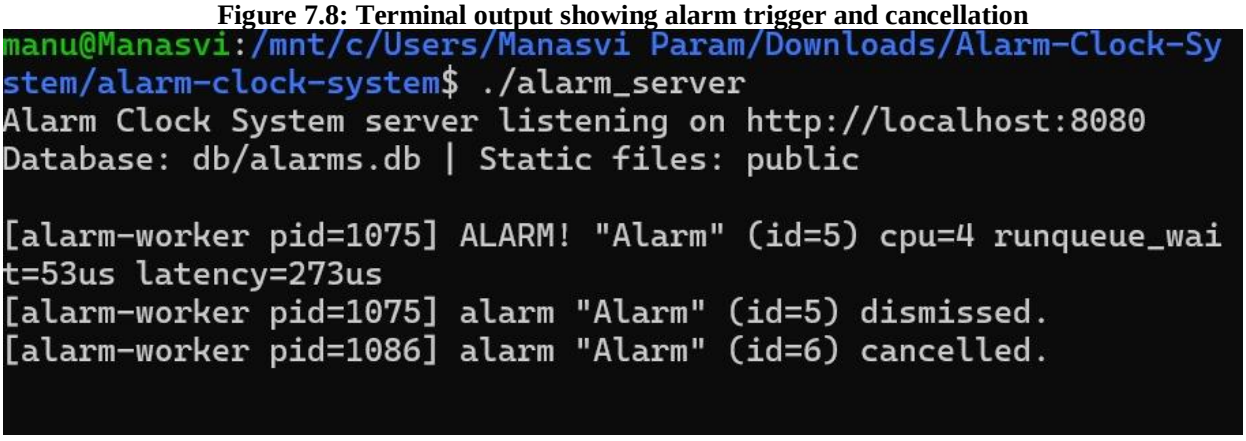


Figure 7.8: Terminal output showing alarm trigger and cancellation



7.5 Demonstration of OS Concepts

The table below connects each operating-system concept to the evidence that was observed while running the system.

Table 7.2: OS concepts demonstrated

OS concept	How it is demonstrated	Evidence
Process creation	One `fork()` per alarm	`ps --forest` shows a child of `alarm_server` for each active alarm (Listing 7.5)
Process states	NEW, RUNNING, WAITING, READY, RUNNING, WAITING, EXITED	Event log (Listing 7.7) and the chips in Figure 7.3
Blocking without busy-waiting	`sigsuspend()`	The live process table shows the worker as Waiting in `sigsuspend`, with 0 ms CPU time and 0% core usage while waiting
Timers	`alarm()` and `SIGALRM`	Alarm fired after the requested 6 s with a measured latency of 153 us
Signal handling	`sigaction()` handlers	Flags set by `on_sigalrm`, `on_sigusr1`, `on_sigusr2`; correct paths taken

OS concept	How it is demonstrated	Evidence
Process control	`kill()` with `SIGUSR1` / `SIGUSR2`	Cancel and dismiss tests (Listing 7.8)
Scheduling	`sched_getcpu()`, `/proc/self/schedstat`	READY to RUNNING on CPU 0 after 10 us in the run queue
Context switches	`/proc/<pid>/status`	The process table shows voluntary and forced switch counts per process
Zombie prevention	`SIGCHLD` handler with `waitpid()`	After the tests only the server process remained
IPC	SQLite shared file plus signals	Worker writes status and events; server and browser read them
Socket programming	TCP server	`curl` and the browser communicate with the C server over port 8080

CHAPTER -8 CONCLUSION

8.1 Summary of the Project

This project designed and implemented an Alarm Clock System that demonstrates how a Linux operating system supports a seemingly simple task. A web page lets the user set an alarm by delay or by clock time. A C server creates one dedicated process for each alarm with `fork()`. The process arms a kernel timer with `alarm()`, sleeps in `sigsuspend()` without using CPU, wakes on `SIGALRM`, and notifies the user through the terminal, a Linux desktop notification and the web page. The user can stop a ringing alarm or cancel a waiting one, which the server implements by sending `SIGUSR2` or `SIGUSR1`. Alarm data and scheduler events are shared through an SQLite database, and the browser shows the process states, CPU core, run-queue delay and timer latency in real time.

The system is written entirely in C, builds without compiler warnings with a simple Makefile, and runs on Ubuntu, Debian and WSL.

8.2 Objectives Achieved

Table 8.1: Objectives and their outcome

Objective	Outcome
Apply process, signal, timer, scheduling and IPC concepts	Achieved: all five are used in working code and demonstrated in Chapter 7
One process per alarm using `fork()`	Achieved: each alarm has its own PID, visible in `ps --forest`
Generate the event with a timer and wait without busy-waiting	Achieved: `alarm()` plus `sigsuspend()`; waiting alarms show 0 ms CPU time
Cancel and dismiss with signals	Achieved: `SIGUSR1` and `SIGUSR2` sent by `kill()`
Shared SQLite database for IPC and persistence	Achieved: WAL mode, busy time-out and conditional status updates
HTTP server in C and a web front end	Achieved: REST-style JSON API, static file serving and single-page UI
Make scheduler activity visible	Achieved: scheduler panels, `/proc` process table and event log

8.3 Results Achieved

Table 8.2: Measured and observed results

Result	Value / observation
Build	Compiles with `-Wall -Wextra` and no warnings; executable about 94 KB

Result	Value / observation
Source size	About 1,400 lines of C, HTML, CSS and JavaScript
Timer latency (6 s alarm)	153 microseconds
Run-queue wait after wake-up	10 microseconds
CPU use while waiting	0 ms CPU time; process blocked in `sigsuspend`
Concurrency	Multiple alarms run as independent processes
Cleanup	No zombie processes after dismiss or cancel; orphans cancelled at start-up
Error handling	Invalid delay, unknown alarm and wrong-state requests are rejected with JSON errors

Limitations and possible enhancements. The server handles one HTTP request at a time and its JSON reader is deliberately minimal, because it only parses the fixed requests sent by this project's own front end. Alarms are not recurring, there is no user authentication or HTTPS, and a restart of the server cancels alarms whose workers no longer exist. Possible improvements are recurring alarms, a multi-user login, re-attachment of workers after a restart, and a concurrent server design for heavier use.

8.4 Skills and Learning Outcomes

- **Systems programming in C:** use of POSIX / Linux system calls, header and module organisation, bounded string and buffer handling, and Makefile-based builds.
- **Process management:** creating, controlling and reaping processes, and understanding parent-child relationships and copy-on-write memory.
- **Signals and timers:** installing handlers, writing async-signal-safe code, using signal masks and avoiding lost-wake-up races.
- **Scheduling insight:** reading the run queue, CPU core, context-switch and wait-channel information from `/proc`.
- **Inter-process communication and concurrency control:** signals, a shared database in WAL mode, and compare-and-set updates.
- **Network programming:** building a TCP / HTTP server and a JSON REST interface from sockets.
- **Web front-end development:** HTML, CSS, JavaScript, the Fetch API and the Web Audio API.
- **Testing and documentation:** verifying behaviour with `curl`, `ps` and the browser, and presenting results with diagrams and screenshots.

CHAPTER -9 REFERENCES

9.1 Books

[1] A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ: Wiley, 2018.

[2] W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. Upper Saddle River, NJ: Addison-Wesley, 2013.

[3] M. Kerrisk, *The Linux Programming Interface*. San Francisco, CA: No Starch Press, 2010.

[4] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Boston, MA: Pearson, 2015.

[5] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Englewood Cliffs, NJ: Prentice Hall, 1988.

[6] R. Love, *Linux Kernel Development*, 3rd ed. Upper Saddle River, NJ: Addison-Wesley, 2010.

9.2 Research Papers

[1] D. M. Ritchie and K. Thompson, "The UNIX time-sharing system," *Communications of the ACM*, vol. 17, no. 7, pp. 365-375, Jul. 1974.

[2] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Dept. of Information and Computer Science, Univ. of California, Irvine, 2000.

[3] C. S. Pabla, "Completely fair scheduler," *Linux Journal*, no. 184, Aug. 2009.

9.3 Linux / GNU Documentation

[1] Linux man-pages project, *fork(2)*, *waitpid(2)*, *kill(2)*, *alarm(2)*, *sigaction(2)*, *sigprocmask(2)*, *sigsuspend(2)*, *signal(7)* and *proc(5)*. https://man7.org/linux/man-pages/

[2] Linux man-pages project, *sched_getcpu(3)* and *clock_gettime(2)*. https://man7.org/linux/man-pages/

[3] The GNU C Library Reference Manual. https://www.gnu.org/software/libc/manual/

[4] GNU Make Manual. https://www.gnu.org/software/make/manual/

[5] GCC, the GNU Compiler Collection: Online documentation. https://gcc.gnu.org/onlinedocs/

9.4 Online Resources

[1] SQLite Documentation: Write-Ahead Logging. https://www.sqlite.org/wal.html

[2] SQLite Documentation: An introduction to the SQLite C interface. https://www.sqlite.org/cintro.html

[3] MDN Web Docs: Fetch API. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

[4] MDN Web Docs: Web Audio API. https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API

[5] Microsoft Learn: Windows Subsystem for Linux documentation. https://learn.microsoft.com/en-us/windows/wsl/

[6] Project source: *Alarm Clock System* (alarm-clock-system), README.md and source files in `src/`, `include/` and `public/`.